Primeiros Passos em Python

Fundamentos de Programação

Bernardo Toninho
IST and INESC-ID

Fundamentos de Programação
2026–2027

Baseado em slides de Luís Caires

Outline

1. Primeiros Passos em Python
2. Algoritmos
3. Mais Exemplos Python
4. Sumário e Bibliografia

Um Programa Python

- **Entrada dos dados** — ler valores do utilizador com `input()`

helloworld.py > ...

```
1   # hello world python program
2
3   # get input data
4   print("Hi, what is your name?")
5   s = input()
6
7   # do some computation
8   s = "Hello " + s + ", nice to meet you... Bye!"
9
10  # output the result
11  print(s)
```

Um Programa Python

- **Entrada dos dados** — ler valores do utilizador com `input()`
- **Processar os dados** — calcular resultados

helloworld.py > ...

```
1   # hello world python program
2
3   # get input data
4   print("Hi, what is your name?")
5   s = input()
6
7   # do some computation
8   s = "Hello " + s + ", nice to meet you... Bye!"
9
10  # output the result
11  print(s)
```

Um Programa Python

- **Entrada dos dados** — ler valores do utilizador com `input()`
- **Processar os dados** — calcular resultados
- **Saída de Resultados** — apresentar resultados com `print()`

helloworld.py > ...

```
1   # hello world python program
2
3   # get input data
4   print("Hi, what is your name?")
5   s = input()
6
7   # do some computation
8   s = "Hello " + s + ", nice to meet you... Bye!"
9
10  # output the result
11  print(s)
```

Um Programa Python

- **Entrada dos dados** — ler valores do utilizador com `input()`
- **Processar os dados** — calcular resultados
- **Saída de Resultados** — apresentar resultados com `print()`
- **Nomes (variáveis)** que referem dados

helloworld.py > ...

```
1   # hello world python program
2
3   # get input data
4   print("Hi, what is your name?")
5   s = input()
6
7   # do some computation
8   s = "Hello " + s + ", nice to meet you... Bye!"
9
10  # output the result
11  print(s)
```

Tipos Numericos Primitivos: `int` e `float`

- `int`: números inteiros, sem limite de tamanho em Python.

```python
# int
x = 42
y = -7
grande = 10 ** 100   # googol!

# float
pi = 3.14159
e  = 2.718
muito_pequeno = 1e-9   # 0.000000001

# Verificar o tipo
type(42)       # <class 'int'>
type(3.14)     # <class 'float'>
```

Tipos Numericos Primitivos: `int` e `float`

- **`int`**: números inteiros, sem limite de tamanho em Python.
- **`float`**: números reais (vírgula flutuante), representação aproximada.

```python
# int
x = 42
y = -7
grande = 10 ** 100   # googol!

# float
pi = 3.14159
e  = 2.718
muito_pequeno = 1e-9   # 0.000000001

# Verificar o tipo
type(42)       # <class 'int'>
type(3.14)     # <class 'float'>
```

Tipos Numericos Primitivos: `int` e `float`

- **`int`**: números inteiros, sem limite de tamanho em Python.
- **`float`**: números reais (vírgula flutuante), representação aproximada.

```python
# int
x = 42
y = -7
grande = 10 ** 100   # googol!

# float
pi = 3.14159
e  = 2.718
muito_pequeno = 1e-9   # 0.000000001

# Verificar o tipo
type(42)       # <class 'int'>
type(3.14)     # <class 'float'>
```

- **Atenção** com floats:

```python
0.1 + 0.2
# 0.30000000000000004
# (nao exatamente 0.3!)

# Para comparar floats:
import math
math.isclose(0.1 + 0.2, 0.3)  # True
```

Porquê?

Floats são representados em binário — alguns decimais não têm representação exacta.

Tipos de Dados: `str` e `bool`

`str` — texto:

```
s1 = "Olá, mundo!"
s2 = 'Python é fixe'
s3 = """texto em
varias linhas"""

len("hello")      # 5
"Olá" + " " + "FP"  # "Olá FP"
"ha" * 3          # "hahaha"
"P" in "Python"   # True
```

Tipos de Dados: `str` e `bool`

`str` — texto:

```python
s1 = "Olá, mundo!"
s2 = 'Python é fixe'
s3 = """texto em
varias linhas"""

len("hello")       # 5
"Olá" + " " + "FP"  # "Olá FP"
"ha" * 3           # "hahaha"
"P" in "Python"    # True
```

`bool` — lógico:

```python
v = True
f = False

# Resultados de comparações
5 > 3     # True
5 == 5    # True
5 != 3    # True
5 >= 6    # False

# bool é subtipo de int!
True + True   # 2
int(False)    # 0
int(True)     # 1
```

Variáveis e Atribuição

- Uma **variável** é um nome que se refere a um valor na memória.

```
1 x = 10           # x refere 10
2 y = x + 5        # y refere 15
3 x = x + 1        # x refere 11
4 nome = "Ana"     # nome refere "Ana"
5 nome = 42        # agora nome refere 42!
6
7 # Atribuicao multipla
8 a, b = 3, 7      # a=3, b=7
9 a, b = b, a      # trocar! a=7, b=3
```

Variáveis e Atribuição

- Uma **variável** é um nome que se refere a um valor na memória.
- **Atribuição** (=): liga um nome a um valor.

```
1 x = 10          # x refere 10
2 y = x + 5       # y refere 15
3 x = x + 1       # x refere 11
4 nome = "Ana"    # nome refere "Ana"
5 nome = 42       # agora nome refere 42!
6
7 # Atribuicao multipla
8 a, b = 3, 7     # a=3, b=7
9 a, b = b, a     # trocar! a=7, b=3
```

Nomes válidos

- Letras, dígitos, _
- Não pode começar com dígito
- Sensível a maiúsculas: `Nome` $\neq$ `nome`
- Convenção: `snake_case`

Variáveis e Atribuição

- Uma **variável** é um nome que se refere a um valor na memória.
- **Atribuição** (=): liga um nome a um valor.
- Em Python, uma variável não tem tipo fixo — pode mudar.

```
1 x = 10          # x refere 10
2 y = x + 5       # y refere 15
3 x = x + 1       # x refere 11
4 nome = "Ana"    # nome refere "Ana"
5 nome = 42       # agora nome refere 42!
6
7 # Atribuicao multipla
8 a, b = 3, 7     # a=3, b=7
9 a, b = b, a     # trocar! a=7, b=3
```

Nomes válidos

- Letras, dígitos, _
- Não pode começar com dígito
- Sensível a maiúsculas: `Nome` ≠ `nome`
- Convenção: `snake_case`

Reservadas

`if`, **`elif`**, **`else`**, **`True`**, **`False`**, **`None`**, …— não podem ser usadas como nomes.

Operadores Aritméticos

Op.	Operação	Exemplo
`+`	adição	`3 + 2` = 5
`-`	subtração	`7 - 4` = 3
`*`	multiplicação	`3 * 4` = 12
`/`	divisão real	`7 / 2` = 3.5
`//`	divisão inteira	`7 // 2` = 3
`%`	resto (módulo)	`7 % 2` = 1
`**`	potência	`2 ** 8` = 256

```
1  # Divisao inteira e resto
2  17 // 5   # 3  (quotiente)
3  17 %  5   # 2  (resto)
4  # Verif.: 17 = 5*3 + 2 OK!
5
6  # Aplicacao: e par?
7  42 % 2 == 0   # True
8  13 % 2 == 0   # False
9
10 # Precedencia: **, depois *, /, //, %
11 # depois +, -  (como na matematica)
12 2 + 3 * 4     # 14 (nao 20!)
13 (2 + 3) * 4   # 20
```

Operadores de Comparação e Lógicos

Comparação (devolvem `bool`):

```
5 == 5    # True
5 != 3    # True
5 >  3    # True
5 >= 5    # True
5 <  3    # False
5 <= 4    # False
```

Operadores de Comparação e Lógicos

Comparação (devolvem `bool`):

```
5 == 5    # True
5 != 3    # True
5 >  3    # True
5 >= 5    # True
5 <  3    # False
5 <= 4    # False
```

Encadeamento (exclusivo de Python!):

```
1 < 5 < 10    # True
0 <= x < 100  # x esta no intervalo
```

Lógicos:

```
True and False   # False
True or  False   # True
not True         # False

# Curto-circuito:
# and: se 1.o e False, nao avalia o 2.o
# or:  se 1.o e True,  nao avalia o 2.o
x = 0
x != 0 and 10/x > 1   # False (seguro!)

# Exemplos combinados:
idade = 20
(18 <= idade <= 65)   # True
```

Exemplos — Soma e Diferença

Especificação

```
program: Give me the first number
user: 20
program: Give me the second number
user: 7
program: The sum is 27
program: The difference is 13
```

Exemplos — Soma e Diferença

Especificação

```
program: Give me the first number
user: 20
program: Give me the second number
user: 7
program: The sum is 27
program: The difference is 13
```

```
1 a = int(input("Give me the first number\n"))
2 b = int(input("Give me the second number\n"))
3 print("The sum is", a + b)
4 print("The difference is", a - b)
```

Exemplos — Comparação de Dois Números

Especificação

```
program: Give me a number A   user: 20
program: Give me a number B   user: 7
program: A wins

program: Give me a number A   user: 10
program: Give me a number B   user: 12
program: B wins

program: Give me a number A   user: 2
program: Give me a number B   user: 2

program: its a tie !
```

Exemplos — Comparação de Dois Números

Especificação

```
program: Give me a number A   user: 20
program: Give me a number B   user: 7
program: A wins

program: Give me a number A   user: 10
program: Give me a number B   user: 12
program: B wins

program: Give me a number A   user: 2
program: Give me a number B   user: 2
program: its a tie !
```

```
1 a = int(input("Give me a number A\n"))
2 b = int(input("Give me a number B\n"))
3 if a > b:
4     print("A wins")
5 elif b > a:
6     print("B wins")
7 else:
8     print("its a tie !")
```

Outline

1. Primeiros Passos em Python
2. Algoritmos
3. Mais Exemplos Python
4. Sumário e Bibliografia

O Que é um Algoritmo?

- Um **algoritmo** é uma sequência finita de instruções precisas que resolve um problema.

Etimologia

A palavra algoritmo vem do nome do matemático persa **Muhammad ibn Musa al-Khwarizmi** (séc. IX), autor do primeiro livro sistemático sobre álgebra.

O Que é um Algoritmo?

- Um **algoritmo** é uma sequência finita de instruções precisas que resolve um problema.
- Propriedades de um bom algoritmo:
  - **Correcto**: produz o resultado esperado
  - **Termina**: não fica preso para sempre
  - **Eficiente**: usa poucos recursos (tempo, memória)
  - **Geral**: funciona para qualquer instância do problema

Etimologia

A palavra algoritmo vem do nome do matemático persa **Muhammad ibn Musa al-Khwarizmi** (séc. IX), autor do primeiro livro sistemático sobre álgebra.

O Que é um Algoritmo?

- Um **algoritmo** é uma sequência finita de instruções precisas que resolve um problema.
- Propriedades de um bom algoritmo:
  - **Correcto**: produz o resultado esperado
  - **Termina**: não fica preso para sempre
  - **Eficiente**: usa poucos recursos (tempo, memória)
  - **Geral**: funciona para qualquer instância do problema
- Algoritmos existem independentemente de qualquer linguagem de programação.

Etimologia

A palavra algoritmo vem do nome do matemático persa **Muhammad ibn Musa al-Khwarizmi** (séc. IX), autor do primeiro livro sistemático sobre álgebra.

Exemplos de Algoritmos do Quotidiano

Algoritmos que conhecemos:

- **Receita de culinária**: sequência de passos para preparar um prato

Exemplos de Algoritmos do Quotidiano

Algoritmos que conhecemos:

- **Receita de culinária**: sequência de passos para preparar um prato
- **Instruções de montagem**: IKEA, LEGO, etc.

Exemplos de Algoritmos do Quotidiano

Algoritmos que conhecemos:

- **Receita de culinária**: sequência de passos para preparar um prato
- **Instruções de montagem**: IKEA, LEGO, etc.
- **Pesquisa num dicionário**: abrir a meio, comparar, ir para a metade correcta (pesquisa binária!)

Exemplos de Algoritmos do Quotidiano

Algoritmos que conhecemos:

- **Receita de culinária**: sequência de passos para preparar um prato
- **Instruções de montagem**: IKEA, LEGO, etc.
- **Pesquisa num dicionário**: abrir a meio, comparar, ir para a metade correcta (pesquisa binária!)
- **Ordenar cartas de jogo**: inserir cada carta na posição correcta (insertion sort!)

Exemplos de Algoritmos do Quotidiano

Algoritmos que conhecemos:

- **Receita de culinária**: sequência de passos para preparar um prato
- **Instruções de montagem**: IKEA, LEGO, etc.
- **Pesquisa num dicionário**: abrir a meio, comparar, ir para a metade correcta (pesquisa binária!)
- **Ordenar cartas de jogo**: inserir cada carta na posição correcta (insertion sort!)
- **GPS**: encontrar o caminho mais rápido entre dois pontos

Reflexão

Quando aprendes a resolver um problema, estás a aprender um algoritmo. Programar é traduzi-lo para uma linguagem que o computador entende.

Duas maneiras de somar de 1 a N:

1. Método passo a passo: somar $1+2+3+\cdots+N$
2. Gauss: $\frac{N(N+1)}{2}$

Ambos correctos, mas eficiências muito diferentes!

De Uma Regra a Código

Problema

Escrever um programa que lê uma idade e decide qual o tipo de bilhete:

- menos de 6 anos: `gratuito`;
- dos 6 aos 17 anos: `jovem`;
- dos 18 aos 64 anos: `adulto`;
- 65 anos ou mais: `senior`.

De Uma Regra a Código

Problema

Escrever um programa que lê uma idade e decide qual o tipo de bilhete:

- menos de 6 anos: `gratuito`;
- dos 6 aos 17 anos: `jovem`;
- dos 18 aos 64 anos: `adulto`;
- 65 anos ou mais: `senior`.

Antes de programar

As condições devem ser **mutuamente exclusivas** e devem cobrir todos os casos relevantes.

De Uma Regra a Código — Uma Solução

Problema

Classificar uma idade como `gratuito`, `jovem`, `adulto` ou `senior`.

De Uma Regra a Código — Uma Solução

Problema

Classificar uma idade como `gratuito`, `jovem`, `adulto` ou `senior`.

```
idade = int(input("Idade? "))

if idade < 0:
    print("idade invalida")
elif idade < 6:
    print("gratuito")
elif idade < 18:
    print("jovem")
elif idade < 65:
    print("adulto")
else:
    print("senior")
```

De Uma Regra a Código — Uma Solução

Problema

Classificar uma idade como `gratuito`, `jovem`, `adulto` ou `senior`.

```
idade = int(input("Idade? "))

if idade < 0:
    print("idade invalida")
elif idade < 6:
    print("gratuito")
elif idade < 18:
    print("jovem")
elif idade < 65:
    print("adulto")
else:
    print("senior")
```

- Cada caso só é testado se os anteriores falharam.

De Uma Regra a Código — Uma Solução

Problema

Classificar uma idade como `gratuito`, `jovem`, `adulto` ou `senior`.

```
idade = int(input("Idade? "))

if idade < 0:
    print("idade invalida")
elif idade < 6:
    print("gratuito")
elif idade < 18:
    print("jovem")
elif idade < 65:
    print("adulto")
else:
    print("senior")
```

- Cada caso só é testado se os anteriores falharam.
- Por isso, depois de `elif idade < 6`, sabemos que `idade >= 0`.

Outline

1. Primeiros Passos em Python
2. Algoritmos
3. Mais Exemplos Python
4. Sumário e Bibliografia

Exemplo: Custo de uma Encomenda

Problema: Dado o subtotal de uma encomenda, calcular o total a pagar.

Regra:

- subtotal negativo: valor inválido;
- subtotal menor que 50 euros: portes de 4,95 euros;
- subtotal de 50 euros ou mais: portes grátis.

```
1  subtotal = float(input("Subtotal: "))
2
3  if subtotal < 0:
4      print("valor invalido")
5  else:
6      portes = 0.0
7      if subtotal < 50:
8          portes = 4.95
9
10     total = subtotal + portes
11     print(f"Portes: {portes:.2f} euros")
12     print(f"Total: {total:.2f} euros")
```

Condicionais: Estrutura `if`/`elif`/`else`

```python
# Forma basica
if condicao:
    # executa se True
    instrucoes

# Com alternativa
if condicao:
    instrucoes_A
else:
    instrucoes_B

# Com multiplas alternativas
if cond1:
    instrucoes_A
elif cond2:
    instrucoes_B
elif cond3:
    instrucoes_C
else:
    instrucoes_D  # nenhuma das anteriores
```

Condicionais: Estrutura `if`/`elif`/`else`

```python
# Forma basica
if condicao:
    # executa se True
    instrucoes

# Com alternativa
if condicao:
    instrucoes_A
else:
    instrucoes_B

# Com multiplas alternativas
if cond1:
    instrucoes_A
elif cond2:
    instrucoes_B
elif cond3:
    instrucoes_C
else:
    instrucoes_D  # nenhuma das anteriores
```

- Apenas **um** ramo é executado (o primeiro cuja condição é verdadeira).

Condicionais: Estrutura `if`/`elif`/`else`

```python
# Forma basica
if condicao:
    # executa se True
    instrucoes

# Com alternativa
if condicao:
    instrucoes_A
else:
    instrucoes_B

# Com multiplas alternativas
if cond1:
    instrucoes_A
elif cond2:
    instrucoes_B
elif cond3:
    instrucoes_C
else:
    instrucoes_D  # nenhuma das anteriores
```

- Apenas **um** ramo é executado (o primeiro cuja condição é verdadeira).
- `elif` = "else if" — evita aninhamento desnecessário.

Condicionais: Estrutura `if`/`elif`/`else`

```python
# Forma basica
if condicao:
    # executa se True
    instrucoes

# Com alternativa
if condicao:
    instrucoes_A
else:
    instrucoes_B

# Com multiplas alternativas
if cond1:
    instrucoes_A
elif cond2:
    instrucoes_B
elif cond3:
    instrucoes_C
else:
    instrucoes_D  # nenhuma das anteriores
```

- Apenas **um** ramo é executado (o primeiro cuja condição é verdadeira).
- `elif` = "else if" — evita aninhamento desnecessário.
- O `else` é opcional — se não for necessário, pode omitir-se.

Boa prática

Nunca escrever `if x == True:` — usar simplesmente `if x:`.

Expressões Booleanas e Tabelas de Verdade

P	Q	P and Q	P or Q	not P
T	T	T	T	F
T	F	F	T	F
F	T	F	T	T
F	F	F	F	T

```python
# Exemplos
x = 5
x > 0 and x < 10   # True
x < 0 or x > 100   # False
not (x == 5)        # False

# De Morgan:
# not (A and B) == (not A) or (not B)
# not (A or B)  == (not A) and (not B)
```

Expressões Booleanas e Tabelas de Verdade

P	Q	`P and Q`	`P or Q`	`not P`
T	T	T	T	F
T	F	F	T	F
F	T	F	T	T
F	F	F	F	T

```python
# Exemplos
x = 5
x > 0 and x < 10   # True
x < 0 or x > 100   # False
not (x == 5)        # False

# De Morgan:
# not (A and B) == (not A) or (not B)
# not (A or B)  == (not A) and (not B)
```

- **Curto-circuito**: Python avalia da esquerda para a direita e para assim que o resultado for determinado.

```python
# Seguro: nao divide por zero
n != 0 and 10/n > 1

# Inseguro (poderia ser):
10/n > 1 and n != 0
# RuntimeError se n == 0!
```

Boas Práticas em Condicionais

Evitar:

```python
if aprovado == True:
    print("ok")

if nota >= 10:
    print("passou")
else:
    if nota >= 8:
        print("oral")
    else:
        print("reprovou")
```

Boas Práticas em Condicionais

Evitar:

```
if aprovado == True:
    print("ok")

if nota >= 10:
    print("passou")
else:
    if nota >= 8:
        print("oral")
    else:
        print("reprovou")
```

Preferir:

```
if aprovado:
    print("ok")

if nota >= 10:
    print("passou")
elif nota >= 8:
    print("oral")
else:
    print("reprovou")
```

Exercício: Ordem dos Casos

Problema

Um programa lê uma temperatura em graus Celsius e imprime:

- `gelo` se a temperatura for menor ou igual a 0;
- `ameno` se estiver abaixo de 25;
- `frio` se estiver abaixo de 15;
- `quente` nos restantes casos.

Questão

Escreva as condições pela ordem correta. O que corre mal se começarmos por `temperatura < 25`?

Uma Solução Possível: Ordem dos Casos

Problema

Classificar uma temperatura em `gelo`, `frio`, `ameno` ou `quente`.

Uma Solução Possível: Ordem dos Casos

Problema

Classificar uma temperatura em `gelo`, `frio`, `ameno` ou `quente`.

```
1  temperatura = float(input("Temperatura? "))
2
3  if temperatura <= 0:
4      print("gelo")
5  elif temperatura < 15:
6      print("frio")
7  elif temperatura < 25:
8      print("ameno")
9  else:
10     print("quente")
```

Uma Solução Possível: Ordem dos Casos

Problema

Classificar uma temperatura em `gelo`, `frio`, `ameno` ou `quente`.

```
1  temperatura = float(input("Temperatura? "))
2
3  if temperatura <= 0:
4      print("gelo")
5  elif temperatura < 15:
6      print("frio")
7  elif temperatura < 25:
8      print("ameno")
9  else:
10     print("quente")
```

- A ordem vai das primeiras fronteiras para os intervalos seguintes.

Exemplo: Divisibilidade e Ordem dos Casos

Problema: Dado um único número, imprimir:

- `Fizz` se múltiplo de 3
- `Buzz` se múltiplo de 5
- `FizzBuzz` se múltiplo de 3 e 5
- o próprio número caso contrário

Antes do código: qual deve ser o primeiro caso testado?

Exemplo: Divisibilidade e Ordem dos Casos

Problema: Dado um único número, imprimir:

- `Fizz` se múltiplo de 3
- `Buzz` se múltiplo de 5
- `FizzBuzz` se múltiplo de 3 e 5
- o próprio número caso contrário

Antes do código: qual deve ser o primeiro caso testado?

```
1  n = int(input("N = "))
2
3  if n % 15 == 0:
4      print("FizzBuzz")
5  elif n % 3 == 0:
6      print("Fizz")
7  elif n % 5 == 0:
8      print("Buzz")
9  else:
10     print(n)
```

Nota

A ordem dos `elif` importa: testar `15` antes de `3` e `5`.

Tipos Python: Tabela Resumo

Tipo	Exemplos	Mutável?	Uso
`int`	`0, 42, -7`	N/A	números inteiros
`float`	`3.14, -0.5`	N/A	números reais
`bool`	`True, False`	N/A	lógica, condições
`list`	`[1,2,3]`	Sim	sequências
`tuple`	`(1,2,3)`	Não	sequências imutáveis
`dict`	`a":1"a":1`	Sim	mapeamentos
`set`	`,2,31,2,3`	Sim	conjuntos
`None`	`None`	N/A	ausência de valor

Tipos Python: Tabela Resumo

Tipo	Exemplos	Mutável?	Uso
`int`	`0, 42, -7`	N/A	números inteiros
`float`	`3.14, -0.5`	N/A	números reais
`bool`	`True, False`	N/A	lógica, condições
`list`	`[1,2,3]`	Sim	sequências
`tuple`	`(1,2,3)`	Não	sequências imutáveis
`dict`	`a":1"a":1`	Sim	mapeamentos
`set`	`,2,31,2,3`	Sim	conjuntos
`None`	`None`	N/A	ausência de valor

- Em FP aprenderemos todos estes tipos ao longo do semestre.

Tipos Python: Tabela Resumo

Tipo	Exemplos	Mutável?	Uso
`int`	`0, 42, -7`	N/A	números inteiros
`float`	`3.14, -0.5`	N/A	números reais
`bool`	`True, False`	N/A	lógica, condições
`list`	`[1,2,3]`	Sim	sequências
`tuple`	`(1,2,3)`	Não	sequências imutáveis
`dict`	`a":1"a":1`	Sim	mapeamentos
`set`	`,2,31,2,3`	Sim	conjuntos
`None`	`None`	N/A	ausência de valor

- Em FP aprenderemos todos estes tipos ao longo do semestre.
- **Hoje**: `int`, `float`, `bool`. Mais tarde: `list`, `tuple`, `dict`, `set`.

O Que Vem a Seguir

- **Aulas 3–4**: estado, memória, composição, controlo aninhado e primeiros programas estruturados.

Fio condutor

Cada novo conceito resolve uma limitação do anterior:

- Escolhas → condicionais
- Repetição → ciclos
- Organização → funções
- Muitos dados → colecções
- Relações → grafos
- Modelos → classes

O Que Vem a Seguir

- **Aulas 3–4**: estado, memória, composição, controlo aninhado e primeiros programas estruturados.
- **Aulas 5–6**: ciclos numéricos, invariantes, funções, contratos e decomposição.

Fio condutor

Cada novo conceito resolve uma limitação do anterior:

- Escolhas → condicionais
- Repetição → ciclos
- Organização → funções
- Muitos dados → colecções
- Relações → grafos
- Modelos → classes

O Que Vem a Seguir

- **Aulas 3–4**: estado, memória, composição, controlo aninhado e primeiros programas estruturados.
- **Aulas 5–6**: ciclos numéricos, invariantes, funções, contratos e decomposição.
- **Aulas 7–8**: tuplos como representação; listas, dicionários e dataclasses.

Fio condutor

Cada novo conceito resolve uma limitação do anterior:

- Escolhas → condicionais
- Repetição → ciclos
- Organização → funções
- Muitos dados → colecções
- Relações → grafos
- Modelos → classes

O Que Vem a Seguir

- **Aulas 3–4**: estado, memória, composição, controlo aninhado e primeiros programas estruturados.
- **Aulas 5–6**: ciclos numéricos, invariantes, funções, contratos e decomposição.
- **Aulas 7–8**: tuplos como representação; listas, dicionários e dataclasses.
- **Aula 9**: `for`/`range`, tuplos, excepções e módulos essenciais.

Fio condutor

Cada novo conceito resolve uma limitação do anterior:

- Escolhas → condicionais
- Repetição → ciclos
- Organização → funções
- Muitos dados → colecções
- Relações → grafos
- Modelos → classes

O Que Vem a Seguir

- **Aulas 3–4**: estado, memória, composição, controlo aninhado e primeiros programas estruturados.
- **Aulas 5–6**: ciclos numéricos, invariantes, funções, contratos e decomposição.
- **Aulas 7–8**: tuplos como representação; listas, dicionários e dataclasses.
- **Aula 9**: `for`/`range`, tuplos, excepções e módulos essenciais.
- **Aulas 11–15**: strings, memória, grafos não ponderados com BFS, ficheiros e dados estruturados.

Fio condutor

Cada novo conceito resolve uma limitação do anterior:

- Escolhas → condicionais
- Repetição → ciclos
- Organização → funções
- Muitos dados → colecções
- Relações → grafos
- Modelos → classes

O Que Vem a Seguir

- **Aulas 3–4**: estado, memória, composição, controlo aninhado e primeiros programas estruturados.
- **Aulas 5–6**: ciclos numéricos, invariantes, funções, contratos e decomposição.
- **Aulas 7–8**: tuplos como representação; listas, dicionários e dataclasses.
- **Aula 9**: `for`/`range`, tuplos, excepções e módulos essenciais.
- **Aulas 11–15**: strings, memória, grafos não ponderados com BFS, ficheiros e dados estruturados.
- **Aulas 16–21**: pesquisa, inserção, ordenação elementar e recursão sobre dados encaixados.

Fio condutor

Cada novo conceito resolve uma limitação do anterior:

- Escolhas → condicionais
- Repetição → ciclos
- Organização → funções
- Muitos dados → colecções
- Relações → grafos
- Modelos → classes

O Que Vem a Seguir

- **Aulas 3–4**: estado, memória, composição, controlo aninhado e primeiros programas estruturados.
- **Aulas 5–6**: ciclos numéricos, invariantes, funções, contratos e decomposição.
- **Aulas 7–8**: tuplos como representação; listas, dicionários e dataclasses.
- **Aula 9**: `for`/`range`, tuplos, excepções e módulos essenciais.
- **Aulas 11–15**: strings, memória, grafos não ponderados com BFS, ficheiros e dados estruturados.
- **Aulas 16–21**: pesquisa, inserção, ordenação elementar e recursão sobre dados encaixados.
- **Aulas 23–25**: programação funcional: funções como valores, compreensões, geradores, pipelines, closures e decoradores.

Fio condutor

Cada novo conceito resolve uma limitação do anterior:

- Escolhas → condicionais
- Repetição → ciclos
- Organização → funções
- Muitos dados → colecções
- Relações → grafos
- Modelos → classes

O Que Vem a Seguir

- **Aulas 3–4**: estado, memória, composição, controlo aninhado e primeiros programas estruturados.
- **Aulas 5–6**: ciclos numéricos, invariantes, funções, contratos e decomposição.
- **Aulas 7–8**: tuplos como representação; listas, dicionários e dataclasses.
- **Aula 9**: `for`/`range`, tuplos, excepções e módulos essenciais.
- **Aulas 11–15**: strings, memória, grafos não ponderados com BFS, ficheiros e dados estruturados.
- **Aulas 16–21**: pesquisa, inserção, ordenação elementar e recursão sobre dados encaixados.
- **Aulas 23–25**: programação funcional: funções como valores, compreensões, geradores, pipelines, closures e decoradores.
- **Aulas 26–27**: classes, TADs, herança, polimorfismo e protocolos essenciais.

Fio condutor

Cada novo conceito resolve uma limitação do anterior:

- Escolhas → condicionais
- Repetição → ciclos
- Organização → funções
- Muitos dados → colecções
- Relações → grafos
- Modelos → classes

Dicas para Programar Bem

1. **Nomes descritivos**: `velocidade_media` não `vm` ou `x`.

```
# MAU comentario (obvio):
i = i + 1  # incrementa i

# BOM comentario (porquê):
i = i + 1  # saltar o delimitador
           # inicial da lista
```

Dicas para Programar Bem

1. **Nomes descritivos**: `velocidade_media` não `vm` ou `x`.
2. **Uma decisão de cada vez**: condições simples são mais fáceis de ler do que expressões gigantes.

```
# MAU comentario (obvio):
i = i + 1  # incrementa i

# BOM comentario (porquê):
i = i + 1  # saltar o delimitador
           # inicial da lista
```

Dicas para Programar Bem

1. **Nomes descritivos**: `velocidade_media` não `vm` ou `x`.
2. **Uma decisão de cada vez**: condições simples são mais fáceis de ler do que expressões gigantes.
3. **Testar sempre**: testa casos normais e casos limite (0, negativos, valores extremos, fronteiras de intervalos).

```
# MAU comentario (obvio):
i = i + 1  # incrementa i

# BOM comentario (porquê):
i = i + 1  # saltar o delimitador
           # inicial da lista
```

Dicas para Programar Bem

1. **Nomes descritivos**: `velocidade_media` não `vm` ou `x`.
2. **Uma decisão de cada vez**: condições simples são mais fáceis de ler do que expressões gigantes.
3. **Testar sempre**: testa casos normais e casos limite (0, negativos, valores extremos, fronteiras de intervalos).
4. **Comentar o porquê**: o código diz o quê; um comentário diz porquê.

```
# MAU comentario (obvio):
i = i + 1  # incrementa i

# BOM comentario (porquê):
i = i + 1  # saltar o delimitador
           # inicial da lista
```

Dicas para Programar Bem

1. **Nomes descritivos**: `velocidade_media` não `vm` ou `x`.
2. **Uma decisão de cada vez**: condições simples são mais fáceis de ler do que expressões gigantes.
3. **Testar sempre**: testa casos normais e casos limite (0, negativos, valores extremos, fronteiras de intervalos).
4. **Comentar o porquê**: o código diz o quê; um comentário diz porquê.
5. **Não repetir condições**: se dois ramos fazem quase o mesmo, provavelmente a estrutura pode ser simplificada.

```
1 # MAU comentario (obvio):
2 i = i + 1  # incrementa i
3
4 # BOM comentario (porquê):
5 i = i + 1  # saltar o delimitador
6            # inicial da lista
```

Princípio DRY

Don't **R**epeat **Y**ourself: código duplicado é código que tens de manter em dois sítios.

Dicas para Programar Bem

1. **Nomes descritivos**: `velocidade_media` não `vm` ou `x`.
2. **Uma decisão de cada vez**: condições simples são mais fáceis de ler do que expressões gigantes.
3. **Testar sempre**: testa casos normais e casos limite (0, negativos, valores extremos, fronteiras de intervalos).
4. **Comentar o porquê**: o código diz o quê; um comentário diz porquê.
5. **Não repetir condições**: se dois ramos fazem quase o mesmo, provavelmente a estrutura pode ser simplificada.
6. **Ler os erros**: a mensagem de erro diz-te a linha e o tipo de problema — lê-a!

```
1 # MAU comentario (obvio):
2 i = i + 1  # incrementa i
3
4 # BOM comentario (porquê):
5 i = i + 1  # saltar o delimitador
6            # inicial da lista
```

Princípio DRY

Don't **R**epeat **Y**ourself: código duplicado é código que tens de manter em dois sítios.

Outline

1. Primeiros Passos em Python
2. Algoritmos
3. Mais Exemplos Python
4. Sumário e Bibliografia

Python: Primeiros Padrões em Resumo

Ler e imprimir:

```
n = int(input("N? "))
print(f"Resultado: {n * 2}")
```

Condicional:

```
if n > 0:
    print("positivo")
elif n < 0:
    print("negativo")
else:
    print("zero")
```

Expressão booleana:

```
0 <= nota <= 20
n != 0 and 10 / n > 1
```

Validação antes do cálculo:

```
if denominador == 0:
    print("divisao impossivel")
else:
    print(numerador / denominador)
```

Ordem dos casos:

```
if idade < 0:
    print("idade invalida")
elif idade < 18:
    print("menor")
else:
    print("adulto")
```

Python: Primeiros Padrões em Resumo

Ler e imprimir:

```
n = int(input("N? "))
print(f"Resultado: {n * 2}")
```

Condicional:

```
if n > 0:
    print("positivo")
elif n < 0:
    print("negativo")
else:
    print("zero")
```

Expressão booleana:

```
0 <= nota <= 20
n != 0 and 10 / n > 1
```

Validação antes do cálculo:

```
if denominador == 0:
    print("divisao impossivel")
else:
    print(numerador / denominador)
```

Ordem dos casos:

```
if idade < 0:
    print("idade invalida")
elif idade < 18:
    print("menor")
else:
    print("adulto")
```

- Estes padrões são os primeiros blocos para escrever programas que decidem corretamente.